Trevon Woods
Sitaram Ayyagari
A.I. at the Edge - ITAI - 3377
02/07/2026

L03 Simulation Documentation

Setup, Training, and Validation

The setup for this lab was pretty straight forward. First, since I already had Python and VS Code already installed on my computer, all I had to do was install TensorFlow and Edge Impulse CLI. Those were pretty routine and took just a couple minutes. Then I had to load the MNIST Dataset and scale the pixel values to float values between 0 and 1. Following that I had to create the model architecture using Keras' Sequential function and loading it with many layers namely, a Conv2D layer with ReLU Activation, MaxPooling, Flatten, and two dense layers. The first dense layer takes the flatten features and sends them through 128 neurons and then a ReLU Activation. The final dense layer is the classification layer that narrows the features down into a vector containing the model's prediction. Finally, we compile the model and fit it to the training and validation data for 5 epochs. Lastly, we convert and save the model to a tensorflow lite file that can be used on edge devices. During training my model achieved a training accuracy of 99.5% and a validation accuracy of 98.6% after 5 epochs.

```
● (venv) (base)             Deploy_AI_on_Simulated_Edge % pip install tensorflow
  Collecting tensorflow
    Downloading tensorflow-2.20.0-cp313-cp313-macosx_12_0_arm64.whl.metadata (4.5 kB)
  Collecting absl-py>=1.0.0 (from tensorflow)
    Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
  Collecting astunparse>=1.6.0 (from tensorflow)
    Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
  Collecting flatbuffers>=24.3.25 (from tensorflow)
    Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
  Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
    Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
  Collecting google_pasta>=0.1.1 (from tensorflow)
    Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
  Collecting libclang>=13.0.0 (from tensorflow)
    Downloading libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl.metadata (5.2 kB)
  Collecting opt_einsum>=2.3.2 (from tensorflow)
    Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
```

```
● (venv) (base)             Deploy_AI_on_Simulated_Edge % npm install -g edge-impulse-cli
  npm warn deprecated uuid@3.4.0: Please upgrade  to version 7 or higher.  Older versions may use Math.random() in certa
  in circumstances, which is known to be problematic.  See https://v8.dev/blog/math-random for details.
  npm warn deprecated tar@7.4.3: Old versions of tar are not supported, and contain widely publicized security vulnerabi
  lities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exo
  rbitant rates) by contacting i@izs.me
  npm warn deprecated gauge@2.7.4: This package is no longer supported.
  npm warn deprecated are-we-there-yet@1.1.7: This package is no longer supported.
  npm warn deprecated npmlog@4.1.2: This package is no longer supported.
  npm warn deprecated har-validator@5.1.5: this library is no longer supported
  npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulner
  abilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at
  exorbitant rates) by contacting i@izs.me
  npm warn deprecated request@2.88.2: request has been deprecated, see https://github.com/request/request/issues/3142
  npm warn deprecated @zeit/dockerignore@0.0.5: "@zeit/dockerignore" is no longer maintained
```

```
on_edge.py > ...
import tensorflow as tf
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train.reshape((-1, 28, 28, 1))
x_test = x_test.reshape((-1, 28, 28, 1))

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
    ])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

with open('model.tflite', 'wb') as f:
    f.write(tflite_model)
```

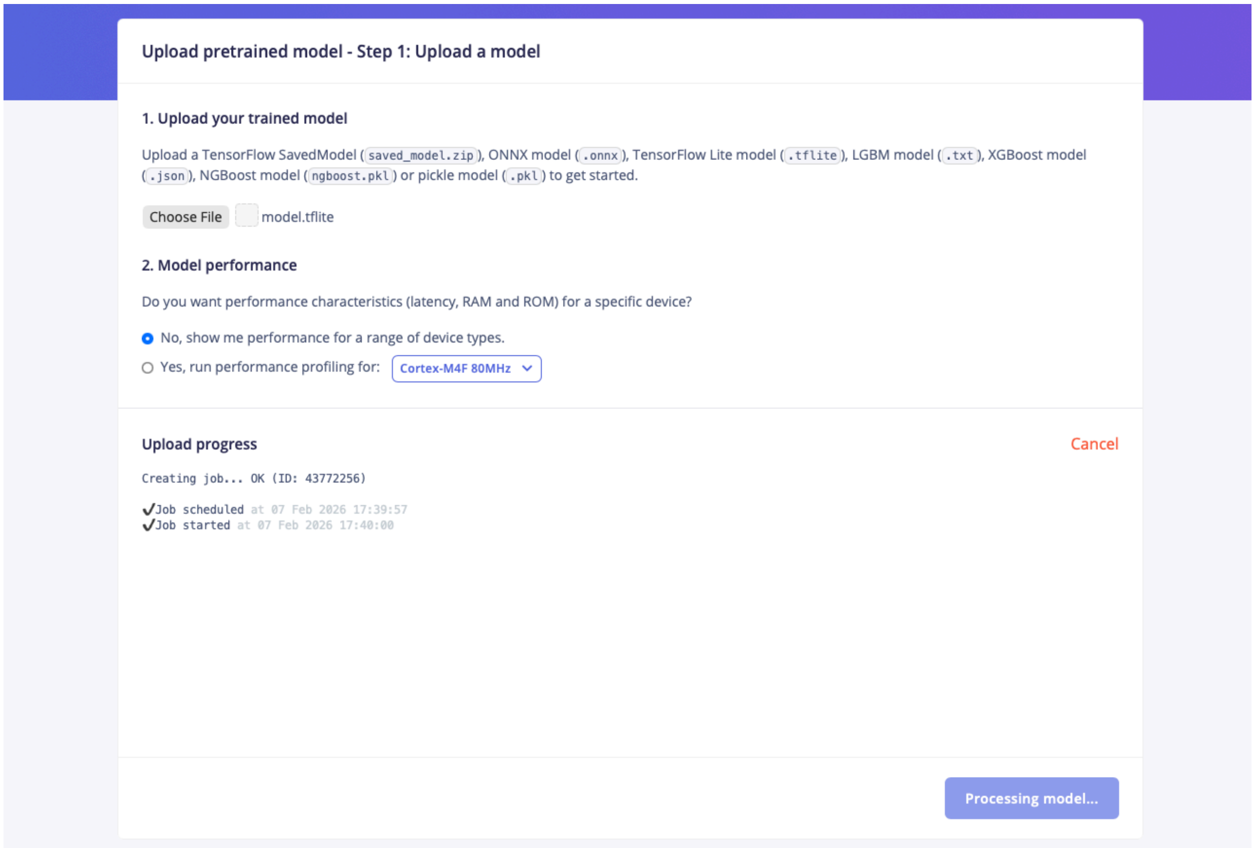
```
(venv) (base)                    Deploy_AI_on_Simulated_Edge % python3 on_edge.py
/Users/              /Documents/AI_on_Edge/Deploy_AI_on_Simulated_Edge/venv/lib/python3.13/site-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9541 - loss: 0.1548 - val_accuracy: 0.9803 - val_loss: 0.0605
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9834 - loss: 0.0543 - val_accuracy: 0.9842 - val_loss: 0.0496
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9895 - loss: 0.0343 - val_accuracy: 0.9866 - val_loss: 0.0402
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9922 - loss: 0.0242 - val_accuracy: 0.9881 - val_loss: 0.0406
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9945 - loss: 0.0166 - val_accuracy: 0.9858 - val_loss: 0.0473
```

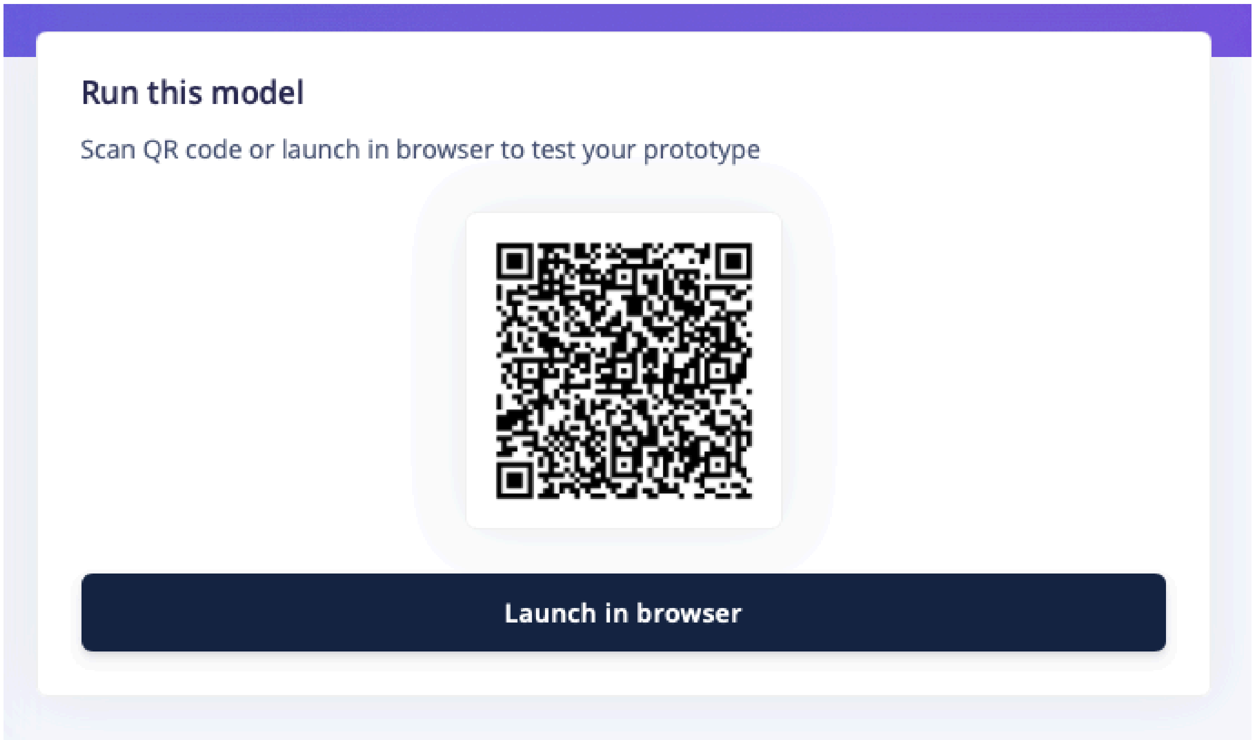
```
model.tflite          from tensorflow.keras.datasets impo
on_edge.py    2
                      (x_train, y_train), (x_test, y_test
                      x_train, x_test = x_train / 255.0,
                      x_train = x_train.reshape((-1, 28,
                      x_test = x_test.reshape((-1, 28, 28

                      model = tf.keras.Sequential([
                          tf.keras.layers.Conv2D(32, (3,
                          tf.keras.layers.MaxPooling2D((2
                          tf.keras.layers.Flatten(),
```

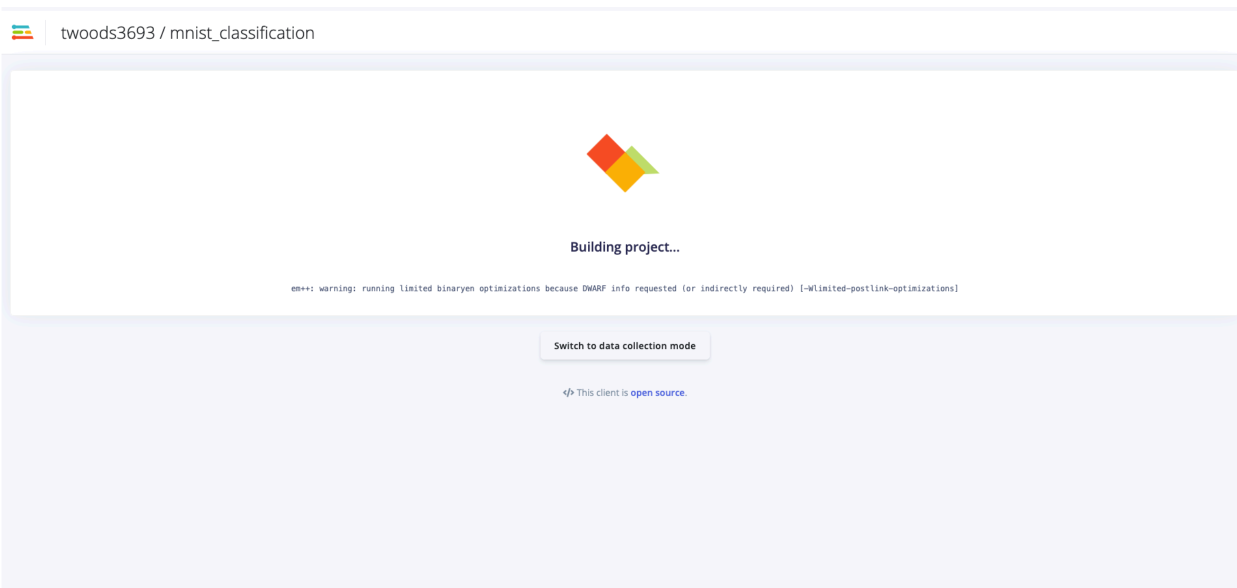
Deployment Process

For the deployment process, I had to upload the .tflite model to Edge Impulse. The CLI command did not work so I had to manually upload the file and ask the system to run performance for a range of device types. The deployment of the model was fairly easy, all I had to do was scan the QR code to use it on an edge device like my phone or open the model in a new browser. There were even features to train the model on Edge Impulse which is something I will try out on my own at some point.


(venv) (base) Deploy_AI_on_Simulated_Edge % edge-impulse-uploader --api-key
model.tflite
Edge Impulse uplo
Follow link (cmd + click)
Endpoints:
API: https://studio.edgeimpulse.com
Ingestion: https://ingestion.edgeimpulse.com
Upload configuration:
Label: Not set, will be inferred from file name
Category: training
Cannot handle this file, only .wav, .cbor, .json, .jpg, .jpeg, .png, .csv, .txt, .mp4, .avi, .parquet supported: model
.tflite

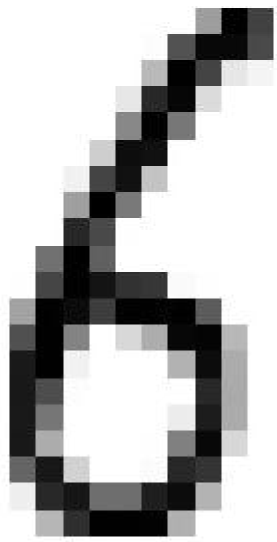

Upload pretrained model - Step 1: Upload a model
1. Upload your trained model
Upload a TensorFlow SavedModel (saved_model.zip), ONNX model (.onnx), TensorFlow Lite model (.tflite), LGBM model (.txt), XGBoost model (.json), NGBoost model (ngboost.pkl) or pickle model (.pkl) to get started.
Choose File
model.tflite
2. Model performance
Do you want performance characteristics (latency, RAM and ROM) for a specific device?
No, show me performance for a range of device types.
Yes, run performance profiling for:
Cortex-M4F 80MHz
Upload progress
Cancel
Creating job... OK (ID: 43772256)
Job scheduled at 07 Feb 2026 17:39:57
Job started at 07 Feb 2026 17:40:00
Processing model...





Testing Results

Testing lacked luster because my model didn't make the right predictions whether on an edge device or through inference on Edge Impulse. I fed the model an image of one of the mnist classes (six) and it predicted that it was a seven. Also during the live testing on an edge device I showed it an image of a seven and it predicted an eight. For some reason the model was predicted one class above whatever it was seeing. Not sure why that is but I will definitely be looking into why the accuracy was so bad. As far as latency goes the model was running well. I didn’t see any lag spikes or slow processing when it made inferences through the edge device camera.



Check model behavior

Upload test data to ensure correct model settings and proper model processing. *(Optional)*

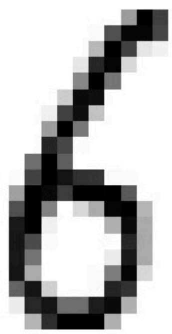
Upload an image

Upload an image to try out your model. The image will be automatically resized to 28x28 (Grayscale).

Choose File 6 Image 25.jpeg

Test sample

Results



INDEX	LABEL	PROBABILITY
0	**class 1**	0.00
1	**class 2**	0.00
2	**class 3**	0.00
3	**class 4**	0.00
4	**class 5**	0.00
5	**class 6**	0.01
6	**class 7**	0.83
7	**class 8**	0.00
8	**class 9**	0.16
9	**class 10**	0.00

Modify model settings if results do not reflect expected model behavior, otherwise save and deploy your model!

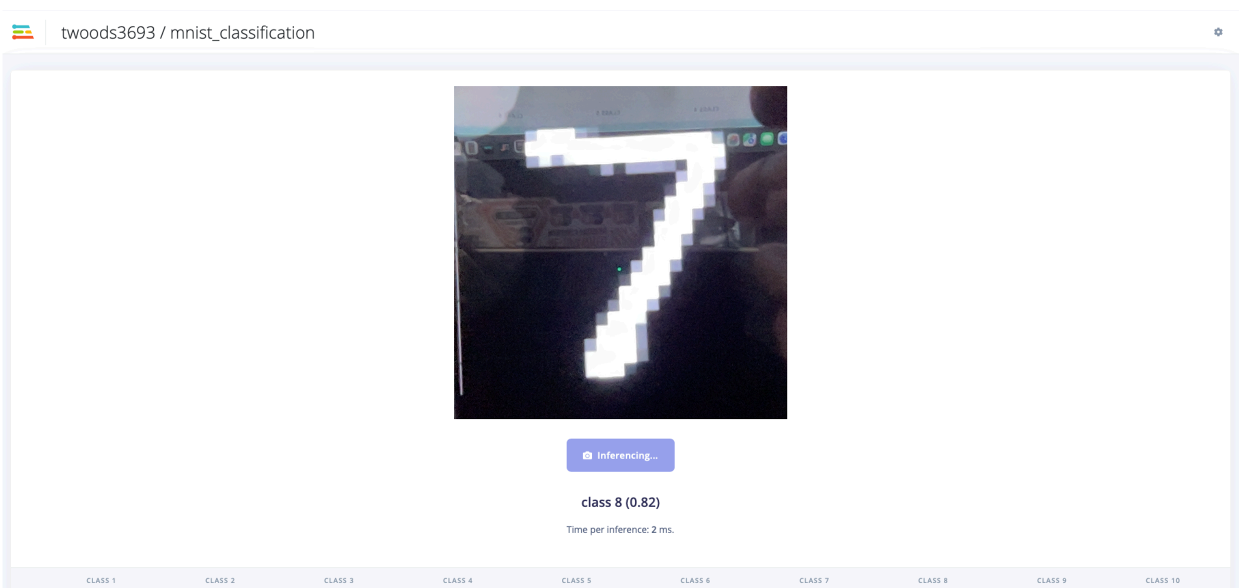
On-device performance

MCUs

		EON COMPILER		TFLITE	
DEVICE	LATENCY	RAM	ROM	RAM	ROM
Low-end MCU ⓘ	15,269 ms.	107.5K	2.7M	130.9K +23.4K	2.7M +18.9K
High-end MCU ⓘ	198 ms.	107.6K	2.7M	131.0K +23.4K	2.7M +27.3K
+ AI accelerator ⓘ	198 ms.	107.6K	2.7M	131.0K +23.4K	2.7M +27.3K

Microprocessors

DEVICE	LATENCY	MODEL SIZE
CPU ⓘ	8 ms.	2.7M
GPU or accelerator ⓘ	2 ms.	2.7M



	CLASS 1	CLASS 2	CLASS 3	CLASS 4	CLASS 5	CLASS 6	CLASS 7	CLASS 8	CLASS 9	CLASS 10
3173	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3172	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3171	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3170	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3169	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3168	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**1.00**	0.00	0.00
3167	0.00	0.00	0.00	0.00	0.00	0.00	0.00	**0.99**	0.00	0.00
3166	0.00	0.00	0.01	0.00	0.01	0.00	0.00	**0.98**	0.00	0.00
3165	0.00	0.00	0.01	0.00	0.13	0.00	0.00	**0.85**	0.00	0.01
3164	0.00	0.02	0.02	0.00	0.26	0.25	0.00	**0.40**	0.00	0.05
3163	0.00	0.03	0.01	0.00	0.29	0.30	0.00	**0.33**	0.00	0.04
3162	0.00	0.03	0.03	0.00	0.26	0.25	0.00	**0.36**	0.00	0.07
3161	0.00	0.02	0.01	0.00	**0.53**	0.17	0.00	0.25	0.00	0.02
3160	0.00	0.00	0.00	0.00	**0.71**	0.09	0.00	0.16	0.00	0.03
3159	0.00	0.00	0.01	0.00	**0.70**	0.16	0.00	0.11	0.00	0.02
3158	0.00	0.02	0.02	0.00	0.40	0.02	0.00	**0.54**	0.00	0.01